Experiment Number : 04 **Date: 04-02026**

ARRAYS

Aim:

To understand and implement array operations in Java.

PRE LAB EXERCISE

QUESTIONS

- ✓ What is an array?

  An array is a collection of elements of the same data type stored in continuous memory locations.

  Each element is accessed using an index number.

  Example:

  int[] a = {10, 20, 30, 40};

- ✓ Why are arrays used?

  Arrays are used because:

- They store multiple values using a single variable name
- They provide easy access to elements using index
- They make data processing faster
- They help in implementing sorting, searching, matrices, etc.

- ✓ What is the difference between array and variable?

Variable	Array
Stores only one value	Stores multiple values
Holds a single data item	Holds a collection of items
Example: `int x = 10;`	Example: `int[] x = new int[5];`
Simple memory usage	Uses continuous memory

IN LAB EXERCISE

Objective:

To perform array operations using simple programs.

PROGRAMS:

1. Program to Read and Print Array Elements

Code:

```
import java.util.Scanner;
public class ReadPrintArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        System.out.println("Enter 5 elements:");
        for(int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
        System.out.println("Array elements are:");
        for(int i = 0; i < 5; i++)
            System.out.print(arr[i] + " ");
    }
}
```

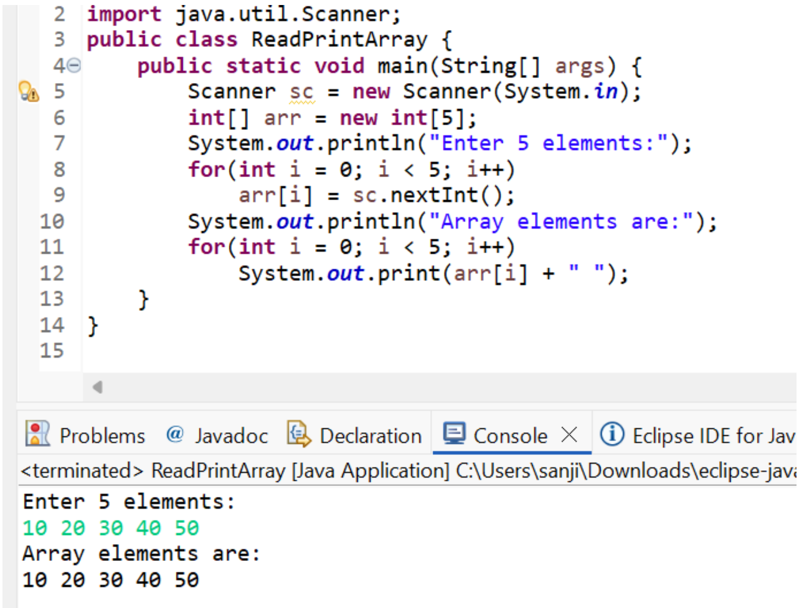
OUTPUT:

Input:
10 20 30 40 50

Output:

Array elements are:

10 20 30 40 50



2. Program to Find Sum of Array Elements

Code:

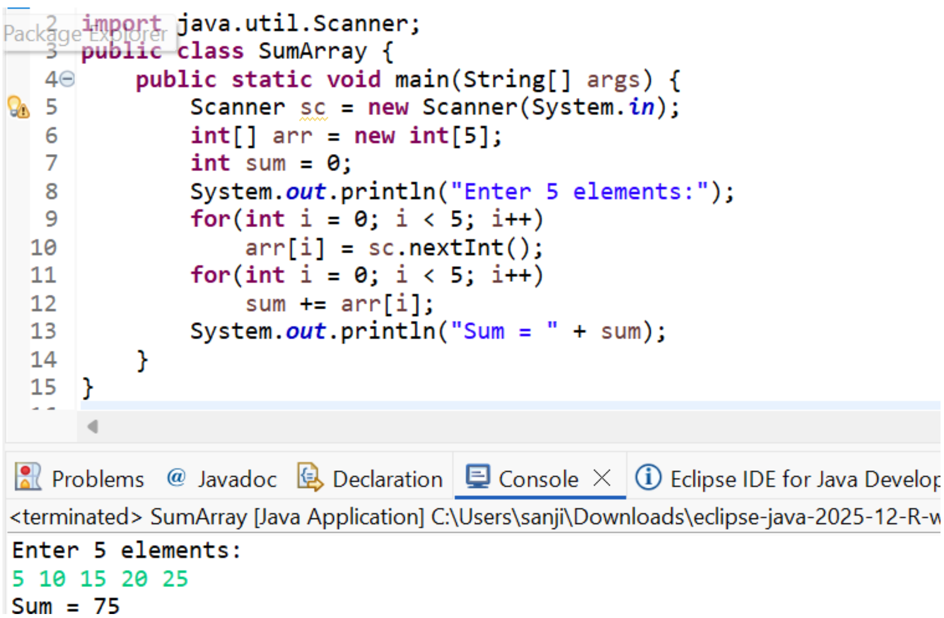
```
import java.util.Scanner;
public class SumArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        int sum = 0;
        System.out.println("Enter 5 elements:");
        for(int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
        for(int i = 0; i < 5; i++)
            sum += arr[i];
        System.out.println("Sum = " + sum);
    }
}
```

OUTPUT:

Input:
5 10 15 20 25

Output:

Sum = 75



3. Program to Find Largest Element in an Array

Code:

```
import java.util.Scanner;

public class LargestElement {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[] arr = new int[5];

        System.out.println("Enter 5 elements:");

        for(int i = 0; i < 5; i++)

            arr[i] = sc.nextInt();

        int max = arr[0];

        for(int i = 1; i < 5; i++)

            if(arr[i] > max)

                max = arr[i];
```

```
        System.out.println("Largest element = " + max);
    }
}
```

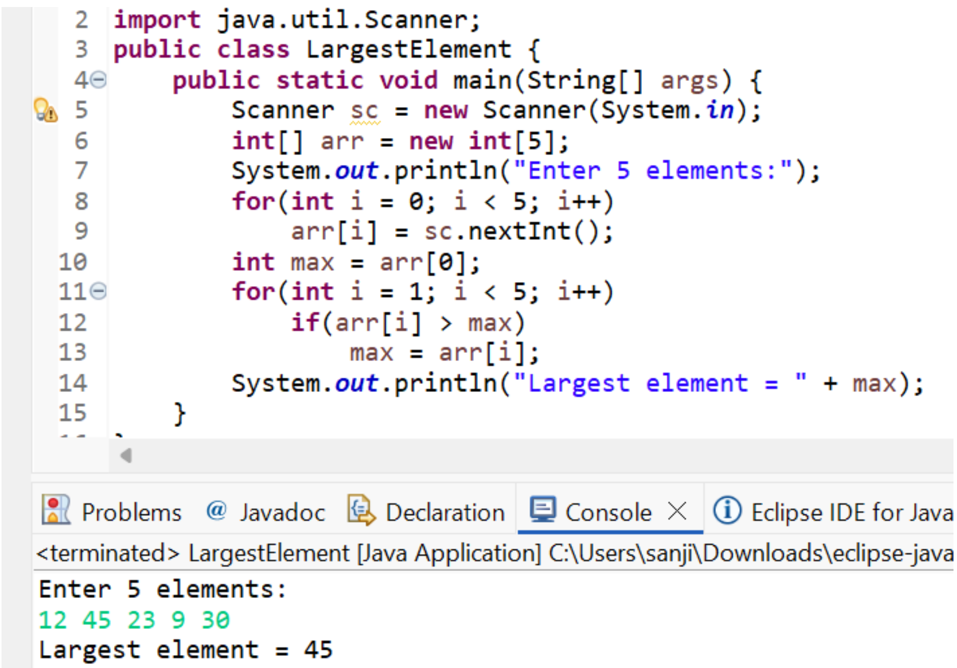
OUTPUT:

Input:

12 45 23 9 30

Output:

Largest element = 45



4. Program to Reverse an Array

Code:

```
import java.util.Scanner;


public class ReverseArray {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        System.out.println("Enter 5 elements:");
```

```
        for(int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
        System.out.println("Reversed array:");
        for(int i = 4; i >= 0; i--)
            System.out.print(arr[i] + " ");
    }
}
```

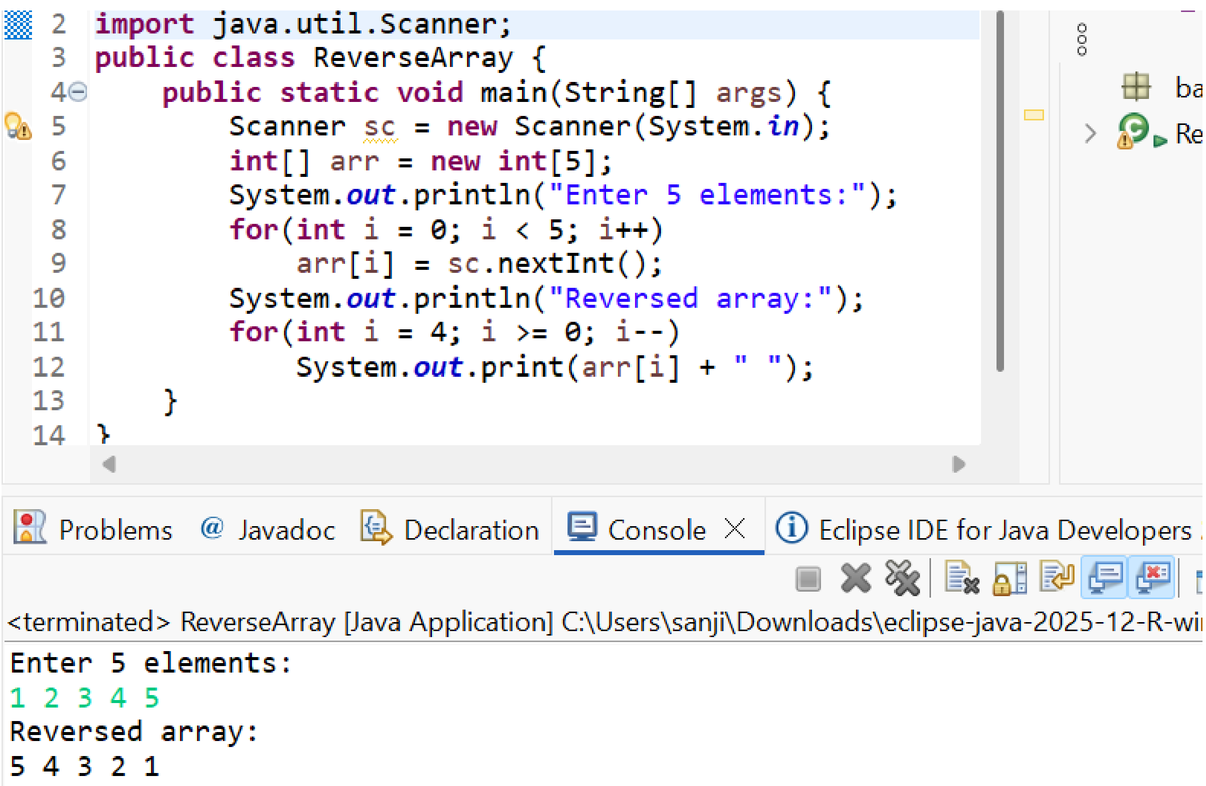
OUTPUT:

Input:
1 2 3 4 5

Output:

Reversed array:

5 4 3 2 1



5. Program to Count Even and Odd Numbers

Code:

```java
import java.util.Scanner;
public class EvenOddCount {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];
        int even = 0, odd = 0;
        System.out.println("Enter 5 elements:");
        for(int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
        for(int i = 0; i < 5; i++) {
            if(arr[i] % 2 == 0)
                even++;
            else
                odd++;
        }

        System.out.println("Even = " + even);
        System.out.println("Odd = " + odd);
    }
}
```

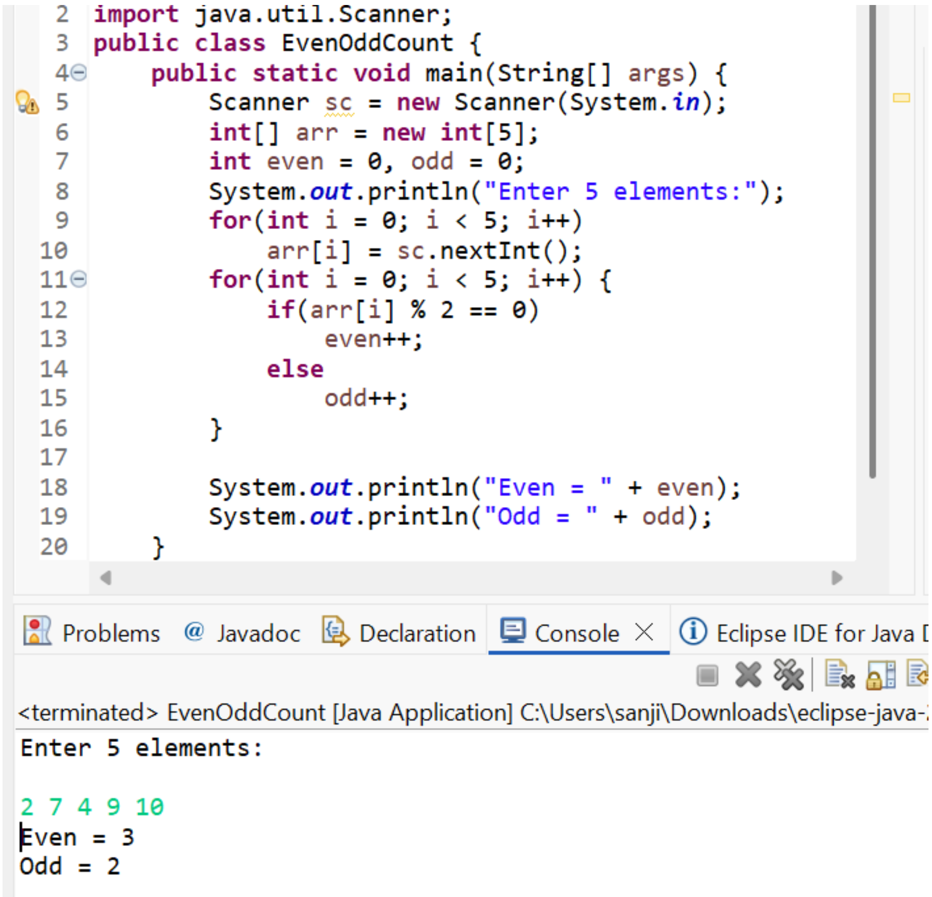
OUTPUT:

Input:
2 7 4 9 10

Output:

Even = 3

Odd = 2



6. Program to Sort Array in Ascending Order

Code:

```
import java.util.Scanner;

public class SortArray {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[] arr = new int[5];

        int temp;

        System.out.println("Enter 5 elements:");

        for(int i = 0; i < 5; i++)

            arr[i] = sc.nextInt();

        for(int i = 0; i < 5; i++) {
```

```
            for(int j = i + 1; j < 5; j++) {
                if(arr[i] > arr[j]) {
                    temp = arr[i];
                    arr[i] = arr[j];
                    arr[j] = temp;
                }
            }
        }
        System.out.println("Sorted array:");
        for(int i = 0; i < 5; i++)
            System.out.print(arr[i] + " ");
    }
}
```

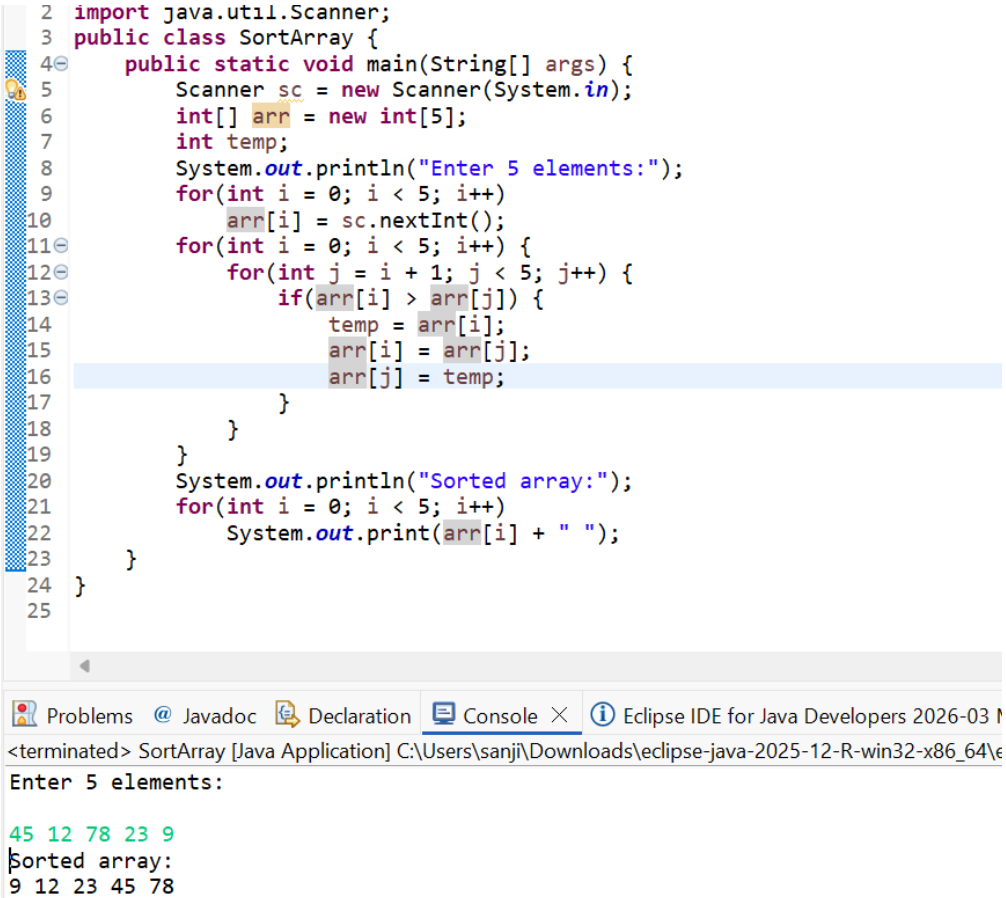
OUTPUT:

Input:
45 12 78 23 9

Output:

Sorted array:

9 12 23 45 78



7. Program to Find Second Largest Element

Code:

```
import java.util.Scanner;


public class SecondLargest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];


        System.out.println("Enter 5 elements:");
        for (int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();
```

```
        int largest = arr[0];
        int second = Integer.MIN_VALUE;

        for (int i = 1; i < 5; i++) {
            if (arr[i] > largest) {
                second = largest;
                largest = arr[i];
            }
            else if (arr[i] > second && arr[i] != largest) {
                second = arr[i];
            }
        }

        System.out.println("Second largest = " + second);
    }
}
```

OUTPUT:

Input:
10 45 23 89 67

Output:

Second largest = 67

```java
import java.util.Scanner;
public class SecondLargest {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] arr = new int[5];

        System.out.println("Enter 5 elements:");
        for (int i = 0; i < 5; i++)
            arr[i] = sc.nextInt();

        int largest = arr[0];
        int second = Integer.MIN_VALUE;

        for (int i = 1; i < 5; i++) {
            if (arr[i] > largest) {
                second = largest;
                largest = arr[i];
            }
            else if (arr[i] > second && arr[i] != largest) {
                second = arr[i];
            }
        }

        System.out.println("Second largest = " + second);
    }
```

Problems @ Javadoc Declaration Console Eclipse IDE for Java Developers 20

<terminated> SecondLargest [Java Application] C:\Users\sanji\Downloads\eclipse-java-2025-12-R-wir

```
Enter 5 elements:
10 45 23 89 67
Second largest = 67
```

8. Program for Matrix Addition (2D Array)

Code:

```java
import java.util.Scanner;

public class MatrixAddition {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[][] a = new int[2][2];

        int[][] b = new int[2][2];

        int[][] sum = new int[2][2];

        System.out.println("Enter elements of matrix A:");

        for(int i = 0; i < 2; i++)

            for(int j = 0; j < 2; j++)

                a[i][j] = sc.nextInt();
```

```
        System.out.println("Enter elements of matrix B:");
        for(int i = 0; i < 2; i++)
            for(int j = 0; j < 2; j++)
                b[i][j] = sc.nextInt();
        for(int i = 0; i < 2; i++)
            for(int j = 0; j < 2; j++)
                sum[i][j] = a[i][j] + b[i][j];
        System.out.println("Sum matrix:");
        for(int i = 0; i < 2; i++) {
            for(int j = 0; j < 2; j++)
                System.out.print(sum[i][j] + " ");
            System.out.println();
        }
    }
}
```

OUTPUT:

Matrix A:

1 2

3 4

Matrix B:

5 6

7 8

Sum matrix:

6 8

10 12

```java
import java.util.Scanner;
public class MatrixAddition {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[][] a = new int[2][2];
        int[][] b = new int[2][2];
        int[][] sum = new int[2][2];
        System.out.println("Enter elements of matrix A:");
        for(int i = 0; i < 2; i++)
            for(int j = 0; j < 2; j++)
                a[i][j] = sc.nextInt();
        System.out.println("Enter elements of matrix B:");
        for(int i = 0; i < 2; i++)
            for(int j = 0; j < 2; j++)
                b[i][j] = sc.nextInt();
        for(int i = 0; i < 2; i++)
            for(int j = 0; j < 2; j++)
                sum[i][j] = a[i][j] + b[i][j];
        System.out.println("Sum matrix:");
        for(int i = 0; i < 2; i++) {
            for(int j = 0; j < 2; j++)
                System.out.print(sum[i][j] + " ");
            System.out.println();
        }
    }
```

Problems @ Javadoc Declaration Console Eclipse IDE for Java Developers 2026-03 M2

```
<terminated> MatrixAddition [Java Application] C:\Users\sanji\Downloads\eclipse-java-2025-12-R-win32-x86_64\eclipse\pl
Enter elements of matrix A:

1 2
3 4
Enter elements of matrix B:
```

```
<terminated> MatrixAddition [Java Application] C:\Users\sanji\Downloads\eclipse-java-2025-
Enter elements of matrix A:

1 2
3 4
Enter elements of matrix B:
5 6
7 8
Sum matrix:
6 8
10 12
```

POST LAB EXERCISE

✓ Why is array indexing usually started from zero instead of one?

Array indexing usually started from zero because:

- Arrays are stored in memory blocks
- The index represents the offset from the base address
- First element is at base address + 0
- This improves performance and memory calculation

Example:

arr[0] refers to the first element

✓ What happens if we try to access an array element outside its declared size?

It causes a runtime error.

In Java, it throws:

ArrayIndexOutOfBoundsException

Example:

int[] a = new int[5];

System.out.println(a[6]); // Error

✓ How does memory allocation differ for static arrays and dynamic arrays?

Static Array	Dynamic Array
Size is fixed at compile time	Size can be decided at runtime
Cannot change size	Can change size (using collections like `ArrayList`)
Faster access	Slightly slower
Example: `int a[5];`	Example: `new int[n];`

✓ Why is searching faster in arrays compared to linked lists?

Searching faster in arrays than linked lists because:

- Arrays support direct access using index
- Time complexity is O(1) for accessing elements
- Linked lists require traversing node by node
- Hence, searching in arrays is faster

✓ What is the difference between contiguous and non-contiguous memory allocation?

Contiguous Memory	Non-Contiguous Memory
Memory blocks are continuous	Memory blocks are scattered
Used in arrays	Used in linked lists
Faster access	Slower access
Easy indexing	No direct indexing

Result:

Thus the array operations were executed successfully.

ASSESSMENT

Description	Max Marks	Marks Awarded
Pre Lab Exercise	**5**	
In Lab Exercise	**10**	
Post Lab Exercise	**5**	
Viva	**10**	
Total	**30**	
Faculty Signature		